Merge Conflict
User's Manual
CS450 – Spring 2025

Table of Contents

OS Commands **1**

- get_date 1
- get_time 2
- help 3
- set_date 4
- set_time 5
- shutdown 6
- version 7
- clear 8
- color 9
- delete_pcb 10
- block_pcb 11
- process_unblock 12
- process_suspend 13
- process_resume 14
- set_pcb_priority 15
- process_show 16
- process_show_ready 17
- process_show_blocked 18
- process_show_all 19
- load_r3 20
- load_p1 21
- load_p2 22
- load_p3 23
- load_p4 24
- load_p5 25
- alarm 26
- mcb_show 27
- mcb_show_all 28
- mcb_show_free 29

mcb_show_alloc 30
allocate_mem 31
free_mem 32

OS Commands

get_date

The *get_date* command allows the user to get the current date within the operating system.

A sample call of get_date would look like:
@ get_date
@ 01:39:25

Which will get the current date of the operating system, which would be in the form of: MM/DD/YY

You might notice that you only receive 2 digits for the year parameter of this function, this is because the operating system will always assume that we are setting the year to be in the 21st century and this always be prefixed with: 20XX.

If the user does change their mind about entering the command, since it is only a one line argument they can just erase the line and continue using the command line as they please. This will avoid any exit checking and simplify the programming.

Syntax and Output:
@ get_date
@ gd

@ MM/DD/YY

get_time

The *get_time* command allows the user to get the current time of the operating system when called.

A sample call of get_time would look like:
@ get_time
@ 14:45:20

The current time of the operating system will be in the format HH:MM:SS in a 24 hour time format (military time).The above example is 2:45:20 pm on a regular 12 hour time format.

If the user does change their mind about entering the command, since it is only a one line argument they can just erase the line and continue using the command line as they please. This will avoid any exit checking and simplify the programming.

Syntax and Output:
@ get_time
@ gt

@ HH:MM:SS

help

The *help* command allows the user to retrieve a complete list of all working commands in the operating system. When called it will print out a list of each available command, its description, and optionally additional parameters if the command requires it.

A sample call of the help command might look like:
@ help

Which will print out a list of all possible commands from the point at which you are in the system.

If the user does change their mind about entering the command, since it is only a one line argument they can just erase the line and continue using the command line as they please. This will avoid any exit checking and simplify the programming.

Syntax:
@ help

set_date

The *set_date* command allows the user to set the current date within the operating system. When called, you will need to enter the new date of the OS on the same line as the current text before submitting the command.

A sample call of set_date would look like:
@ set_date 03/11/19

Which will set the current date of the operating system to the requested command, which would be: 03/11/19

You might notice that you only enter 2 digits for the year parameter of this function, this is because the operating system will always assume that we are setting the year to be in the 21st century and this always be prefixed with: 20XX.

If the user does change their mind about entering the command, since it is only a one line argument they can just erase the line and continue using the command line.

Syntax:
@ set_date MM/DD/YY

set_time

The *set_time* command allows the user to set the current time of the operating system. When called, you will need to enter the desired time of the operating system on the same line as the command call.

A sample call of set_time would look like:
@ set_time 12:00:15

Which will set the current time of the operating system to 12:00:15 in a 24 hour time format, which will be 12pm.

For this command, you will enter the parameters of the time you wish to set on the same line as the command call, this will negate any issues of exit checking and will instead execute the command when the entire line is finished and submitted in the terminal.

Syntax:
@ get_time HH:MM:SS

shutdown

The shutdown sends the shutdown signal to the program. Before executing the shutdown signal, the program will ask the user to confirm their decision with a 'y' or 'n' submission in the terminal.

Example Syntax for calling shutdown command:

@ shutdown
@ sd

If you type 'y', the program will exit and you will no longer be able to operate the system. If you type 'n', the program will continue running and you will be able to type other commands such as help, get_time, etc. If the user types anything else, the program will ask the user to retry their confirmation.

Example Syntax for shutdown confirmation:

@ Are you sure you want to shutdown? (y or n)
@ dsfc
@ Please retype your response: (y or n)
@ y

version

The version command provides the user with the current version of MPX and the compilation date. This command takes no inputs from the user.

If the user does change their mind about entering the command, since it is only a one line argument they can just erase the line and continue using the command line as they please. This will avoid any exit checking and simplify the programming.

Syntax and Output:

@ version

@ Version #.#
@ Compilation Date: MM/DD/YYYY

clear

This function will clear the contents of what the terminal window is viewing when executed and return the pointer to the "home" position (which is the top left of the terminal window).

Sample Command Input/Output:

@ clear

color

This function can be called to change the color of the terminal symbol in the system. It can be changed to: green, blue, yellow, magenta, or cyan.

This function may be cancelled at any point during its execution by typing ‘cancel’ if the user decides to abort the change.

Syntax and Output:

@ color

Color options: green, yellow, blue, magenta, cyan
Please enter the color to change to (or write 'cancel' to cancel):
@ green
Response entered: green
Color changed to: green
@

delete_pcb

This user function gives the user the ability to delete any process control block that they may have created/edited at any point while using the system. The only parameter that the user MUST provide during the call is the name of the process in which they want to delete out of the queue.

A call to the function can be made in one of two ways:
@ pd
@ process_delete

The function will prompt the user for the name parameter after being called in the terminal.

Syntax and Output:

```
@ pd
Please enter the name of the process to delete:
@ name
Deleting PCB: name
PCB Deleted: name
==========================
Ready Processes:
==========================
No processes in ready queue
```

block_pcb

This function edits a process specified by its name and changes its state from what it is currently at to a blocked state. And changes its spot in the queues appropriately.

A call to the function can be made in one of two ways:
@ pb
@ process_block

After calling the function, the system will prompt the user for the name of the process in which they would like to block.

Syntax and Output:
@ pb
Please enter the name of the process to block:
@ name
Blocking PCB: name
PCB Blocked:
Name: name
Class: User
State: Blocked, Suspended
Priority: 2

process_unblock

This function edits a process specified by its name and changes its state from what it is currently at to a blocked state. And changes its spot in the queues appropriately.

A call to the function can be made in one of two ways:
@ pub
@ process_unblock

After calling the function, the system will prompt the user for the name of the process in which they would like to block.

Syntax and Output:
@ pub
Please enter the name of the process to unblock:
@ name
Unblocking PCB: name
PCB Unblocked:
Name: name
Class: User
State: Ready, Suspended
Priority: 2

process_suspend

This function edits a process specified by its name and changes its state from what it is currently at to a suspended state. And changes its spot in the queues appropriately.

A call to the function can be made in one of two ways:
@ psus
@ process_suspend

After calling the function, the system will prompt the user for the name of the process in which they would like to suspend.

Since processes are set to the suspended state as default, you will not be able to create a process and immediately suspend it.

Syntax and Output:
@ psus
Please enter the name of the process to suspend:
@ name
Process is already suspended: name

process_resume

This function edits a process specified by its name and changes its state from what it is currently at to a not suspended state. And changes its spot in the queues appropriately.

A call to the function can be made in one of two ways:
@ pres
@ process_resume

After calling the function, the system will prompt the user for the name of the process in which they would like to un-suspend.

Syntax and Output:
@ pres
Please enter the name of the suspended process to resume:
@ name
Resuming PCB: name
PCB Resumed:
Name: name
Class: User
State: Ready, Not Suspended
Priority: 3

set_pcb_priority

This function allows the user to change the priority of any existing process control block in the system at the time of the command execution. The user will be asked for the name of the process in which they would like to edit, and then they will be prompted for the priority they would like to set for the named process.

A call to the function can be made in one of two ways:
@ pp
@ process_priority

This function can be canceled at any point after calling it if the user changes their mind at any point before it executes. All the user has to do is type cancel and the command will not execute.

Syntax and Output:
@ pp
Please enter the name of the process to update:
@ name
@ Please enter the new priority [0-9] of the process (or write 'cancel' to cancel):
@ 7
Reponse entered: 7
Setting priority for PCB: name to 7
PCB Updated:
Name: name
Class: User
State: Ready, Not Suspended
Priority: 7

process_show

This command shows the user any currently active process in the system. When the command is entered, the system will prompt the user for the name of the process that they would like to be shown. After entering the name, the system will print out the info contained within the specified process.

A call to the function can be made in one of two ways:

@ ps

@ process_show

This function can be canceled at any point after calling it if the user changes their mind at any point before it executes. All the user has to do is type cancel and the command will not execute.

Syntax and Output:

@ ps

Please enter the name of the process to show:

@ name

Locating PCB: name...

Name: name

Class: User

State: Ready, Not Suspended

Priority: 7

process_show_ready

This command can be called by the user to show all ready processes active within the system as well as all of its fields. There are no parameters that need to be specified in order for this command to function properly.

A sample call to the function can be made in one of two ways:

@ psr

@ process_show_ready

Syntax and Output:

@ psr

```
==========================
Ready Processes:
==========================
Name: proc1
Class: User
State: Ready, Suspended
Priority: 3
Name: proc2
Class: Kernel
State: Ready, Suspended
Priority: 7
```

process_show_blocked

This command can be called by the user to show all blocked processes active within the system as well as all of its fields. There are no parameters that need to be specified in order for this command to function properly.

A sample call to the function can be made in one of two ways:

@ psb

@ process_show_blocked

Syntax and Output:

@ psb

```
==========================
Blocked Processes:
==========================
Name: proc2
Class: Kernel
State: Blocked, Suspended
Priority: 7
```

process_show_all

This command can be used to show all existing processes within the system at the time that it is called. It shows the process name, class, priority, suspended status, and state. There are no parameters for this function call.

A sample call to the function can be made in one of two ways:

@ psa

@ process_show_all

Syntax and Output:

@ psa

```
==========================
Ready Processes:
==========================
Name: proc1
Class: User
State: Ready, Suspended
Priority: 3
==========================
Blocked Processes:
==========================
Name: proc2
Class: Kernel
State: Blocked, Suspended
Priority: 7
```

load_r3

This command is used to load 5 test processes that will load into the system and be queued in a non-suspended ready state with a chosen name and priority. This command helps to ensure the working nature of the context in a process. This command will only load processes that have not been created.

A sample call to the function can be made in one of two ways:

@ load_r3

@ lr3

Syntax and Output:

@ lr3

```
Finished Loading: Process 1
Finished Loading: Process 2
Finished Loading: Process 3
Finished Loading: Process 4
Finished Loading: Process 5
```

load_p1

This command is used to load a *ready*, *suspended* test process named “Process 1” of your desired priority into the suspended queue of the system.

A sample call to the function can be made in one of two ways:

@ load_p1

@ lp1

Syntax and Output:

@ lp1

@ Please enter the new priority [0-9] of the process (or write 'cancel' to cancel):

@ 1

Reponse entered: 1

Finished Loading as Suspended: Process 1

Name: Process 1

Class: User

State: Ready, Suspended

Priority: 1

load_p2

This command is used to load a *ready*, *suspended* test process named “Process 2” of your desired priority into the suspended queue of the system.

A sample call to the function can be made in one of two ways:

@ load_p2

@ lp2

Syntax and Output:

@ lp2

@ Please enter the new priority [0-9] of the process (or write 'cancel' to cancel):

@ 1

Reponse entered: 1

Finished Loading as Suspended: Process 1

Name: Process 1

Class: User

State: Ready, Suspended

Priority: 1

load_p3

This command is used to load a *ready*, *suspended* test process named “Process 3” of your desired priority into the suspended queue of the system.

A sample call to the function can be made in one of two ways:

@ load_p3

@ lp3

Syntax and Output:

@ lp3

@ Please enter the new priority [0-9] of the process (or write 'cancel' to cancel):

@ 1

Reponse entered: 1

Finished Loading as Suspended: Process 1

Name: Process 1

Class: User

State: Ready, Suspended

Priority: 1

load_p4

This command is used to load a *ready*, *suspended* test process named “Process 4” of your desired priority into the suspended queue of the system.

A sample call to the function can be made in one of two ways:

@ load_p4

@ lp4

Syntax and Output:

@ lp4

@ Please enter the new priority [0-9] of the process (or write 'cancel' to cancel):

@ 1

Reponse entered: 1

Finished Loading as Suspended: Process 1

Name: Process 1

Class: User

State: Ready, Suspended

Priority: 1

load_p5

This command is used to load a *ready*, *suspended* test process named “Process 5” of your desired priority into the suspended queue of the system.

A sample call to the function can be made in one of two ways:

@ load_p5

@ lp5

Syntax and Output:

@ lp5

@ Please enter the new priority [0-9] of the process (or write 'cancel' to cancel):

@ 1

Reponse entered: 1

Finished Loading as Suspended: Process 1

Name: Process 1

Class: User

State: Ready, Suspended

Priority: 1

alarm

This command can be used to set a running timer that runs in the background of your system and will be triggered when the timer is up.

A sample call to the function can be made in one of two ways:

@ alarm

@ a

Syntax and Output:

@ alarm

Please enter the message for the alarm (100 chars. max):

@ alarm_1

Please enter the time for the alarm (HH:MM:SS):

@ 17:34:40

Creating alarm...

Message: alarm_1

Time: 17:34:40

Alarm created successfully.

mcb_show

This command can be used to locate an existing memory control block in memory by using its address. After location it will display all attributes related to the block.

A sample call to the function can be made in one of two ways:

@ mcb_show

@ ms

Syntax and Output:

```
@ ms
Please enter the address of the MCB to show:
@ d002107
Locating MCB: d002107...

MCB: 0xd002107
Size: 2996
Status: 1
Name: MCB_START_ADDR
```

mcb_show_all

This command can be used to show the current working list of the heap memory in the system, showing all currently free and allocated blocks.

A sample call to the function can be made in one of two ways:

@ mcb_show_all

@ msa

Syntax and Output:

@ msa

```
Showing all MCBs...

==========================
Allocated MCBs:
==========================
No allocated MCBs found.

==========================
Free MCBs:
==========================
[FREE] MCB: 0xd000014
Size: 50000
Status: 0
```

mcb_show_free

This command can be used to show all free MCBs that are in the heap, as well as all attributes associated with them.

A sample call to the function can be made in one of two ways:

@ mcb_show_free

@ msf

Syntax and Output:

@ msf

```
Showing free MCBs...


==========================
Free MCBs:
==========================
[FREE] MCB: 0xd002107
Size: 41709
Status: 0
Name: MCB_START_ADDR
```

mcb_show_alloc

This command can be used to show all allocated MCBs that are in the heap, as well as all attributes associated with them.

A sample call to the function can be made in one of two ways:

@ mcb_show_alloc

@ msl

Syntax and Output:

@ msl

```
Showing allocated MCBs...

============================
Allocated MCBs:
============================
[ALLOCATED] MCB: 0xd0010ef
Size: 4092
Status: 1
Name: PCB_STACK


[ALLOCATED] MCB: 0xd002107
Size: 2996
Status: 1
Name: MCB_START_ADDR
```

allocate_mem

This command can be used to allocate a memory block of any desired size less than or equal to the size of the heap.

A sample call to the function can be made in one of two ways:

@ allocate_mem

@ am

Syntax and Output:

```
@ am
Please enter the size of the MCB to create:
@ 3000
Start address intialized: 0xd002107
Size initialized: 2996
Start address intialized: 0xd002cd7
Size initialized: 38709
```

free_mem

This command can be used to free any block of memory within the heap of the system.

A sample call to the function can be made in one of two ways:

@ free_memory

@ fm

Syntax and Output:

@ fm

Please enter the address of the memory to free (In Hexadecimal):

@ d002107

Freeing Memory at location 0xd002107...

Memory Freed Successfully